

Recherche textuelle : Boyer-Moore-Horspool

1. Principe et applications

La **recherche textuelle** consiste à trouver la 1^{re} **occurrence** d'un **motif** (chaîne de longueur **p**) dans un **texte** (chaîne de longueur **n**).

- **Applications concrètes**
 - **Bio-informatique** : recherche d'une séquence d'ADN (ATCG...) dans un génome entier — les textes peuvent comporter des milliards de caractères, l'efficacité de l'algorithme est donc *cruciale*.
 - **Traitement de texte** : Ctrl+F dans un éditeur ou un navigateur.
 - **Sécurité réseau** : détection de signatures de virus ou d'intrusions dans des flux de données.
- **Principe commun** : déplacer une « fenêtre d'observation » de longueur **p** (celle du **motif**) en dessous du **texte**.
- **Notations**

Variable	Signification
texte	Chaîne dans laquelle on cherche (longueur n)
motif	Chaîne à trouver (longueur p)
i	Position de départ de la fenêtre dans texte (varie de 0 à n-p)
j	Position courante dans motif

On compare toujours **motif[j]** avec **texte[i + j]** : **i** positionne la fenêtre, **j** indexe le caractère dans le motif.

2. Algorithme naïf

Principe : aligner le **motif** à gauche sous le **texte**, comparer lettre à lettre **de la gauche vers la droite**. En cas d'échec de la comparaison, décaler d'**un seul cran** le **motif** vers la droite et recommencer.

```
def rechercher_naif(texte, motif):
    """Cherche la position de la première occurrence de motif dans texte.
    Renvoie la position en cas de succès, ou -1 en cas d'échec."""
    n, p = len(texte), len(motif)
    i = 0
    while i + p <= n:
        j = 0
        while j < p and texte[i + j] == motif[j]:
            j += 1
        if j == p:
            return i          # Occurrence trouvée en position i
        i += 1                # Décalage d'un seul cran du motif vers la droite
    return -1                # Motif absent
```

C'est la faiblesse de l'algorithme naïf : même si un caractère bloquant est très éloigné dans le `motif`, on ne progresse que d'un cran.
L'algorithme de Boyer-Moore-Horspool vise à *maximiser* ce décalage.

3. Algorithme de Boyer-Moore-Horspool

3.1. Deux idées clés

- 1. **Comparaison de droite à gauche** : on commence par comparer la *dernière* lettre du `motif` avec le `texte`.
- 2. **Saut variable** : en cas d'échec, on décale le `motif` de plusieurs crans d'un coup, grâce à une **table du mauvais caractère** calculée *avant* la recherche.

3.2. Table du mauvais caractère

On construit une « table » `saut` à partir du `motif`, en parcourant les **p-1 premiers caractères** (tous *sauf le dernier*) :

`saut[c] = p - 1 - derniere_position(c)` pour `c` décrivant `motif[0:p-1]`.

L'idée : en dessous du `texte`, faire glisser le `motif` vers la droite de sorte à aligner le caractère bloquant du `texte` avec son homologue dans le `motif` — comme on recule un calque pour faire coïncider deux repères. Si ce caractère est absent du `motif`, on saute d'un coup toute sa longueur `p` !

En pratique, on recourt à un dictionnaire Python (et il est inutile de coder une fonction `derniere_position(c)`) :

```
p = len(motif)
saut = {}
for i in range(p - 1):      # SAUF le dernier caractère !
    saut[motif[i]] = p - 1 - i
```

Remarque : construction de la table ▼

En parcourant `motif` de la gauche vers la droite, on est *certain* qu'on obtient bien la **dernière position** de chaque caractère dans les `p-1` premiers caractères de `motif` (son occurrence la plus à droite). Si un caractère est répété dans `motif`, son décalage est recalculé à chaque occurrence. On a donc la garantie d'avoir un saut minimal sans risquer de manquer une occurrence.

⚠ Pourquoi s'arrêter à p-1 ?

On exclut *délibérément* le dernier caractère du `motif` : c'est lui qui est aligné avec le caractère bloquant du `texte`. L'inclure provoquerait un décalage nul (saut de 0), ce qui bloquerait l'algorithme.
En outre, si ce dernier caractère est bien bloquant, on sait que le `motif` peut être décalé de *toute sa longueur*.

✏ Exemple : motif dab ▼

On a `p = 3`.

Caractère	Dernière position dans <code>motif[0:2]</code>	Saut
d	0	$3 - 1 - 0 = 2$
a	1	$3 - 1 - 1 = 1$
Tout autre	absent	3 (= p)

Comme indiqué, `b` est le dernier caractère : il est exclu de la table.

3.3. Déroulement

- 1. On aligne le motif à gauche.
- 2. On compare **de droite à gauche** (`j` part de `p-1`).
- 3. Coïncidence complète (`j < 0`) → occurrence trouvée en position `i`.
- 4. Sinon, on lit `c = texte[i + p - 1]` (le caractère du texte sous la **dernière lettre** du motif) et on décale :
`i += saut.get(c, p)`.

```
def horspool(texte, motif):
    n, p = len(texte), len(motif)
    if p == 0:
        return 0

    saut = {}
    for i in range(p - 1):
        saut[motif[i]] = p - 1 - i

    i = 0
    while i + p <= n:
        j = p - 1
        while j >= 0 and texte[i + j] == motif[j]:
            j -= 1
        if j < 0:
            return i                # Occurrence trouvée !
        i += saut.get(texte[i + p - 1], p)

    return -1
```

Remarque: `dico.get(clef, valeur_par_défaut)` ▼
`saut.get(c, p)` renvoie `saut[c]` si `c` est une clef du dictionnaire, et `p` sinon — sans provoquer d'erreur. Équivalent explicite :
`saut[c] if c in saut else p`.

Exemple : ▼

Texte: `abracadabra`.
Motif: `dab` (de longueur `p=3`).
Table: `saut = {'d': 2, 'a': 1}` (tout autre caractère : saut de 3).

Étape	i	Fenêtre	Comparaison droite→gauche	Caractère sous dernière lettre	Décision
1	0	abr	<code>r ≠ b</code> → échec immédiat	<code>r</code> → absent → saut 3	<code>i = 3</code>
2	3	aca	<code>a ≠ b</code> → échec immédiat	<code>a</code> → saut 1	<code>i = 4</code>
3	4	cad	<code>d ≠ b</code> → échec immédiat	<code>d</code> → saut 2	<code>i = 6</code>
4	6	dab	Coïncidence complète ✓	—	Renvoyer 6

Dans cet exemple, aucune coïncidence partielle n'a lieu : la comparaison droite-à-gauche échoue dès le premier caractère examiné (le dernier du motif), et on consulte immédiatement la table. L'étape 4 illustre le cas d'une coïncidence complète, où la comparaison réussit de droite à gauche jusqu'à épuisement du motif.

4. Boyer-Moore complet : la table du bon suffixe

Pour aller plus loin (hors programme, d'après [ce document d'accompagnement](#)) : l'algorithme de **Boyer-Moore complet** (1977) ajoute une seconde heuristique à celle du mauvais caractère : la **table du bon suffixe**.

Remarque : heuristique ? ▼

C'est une méthode de résolution de problème qui ne garantit pas la solution optimale, mais qui permet d'arriver à une solution satisfaisante rapidement, en s'appuyant sur des « règles pratiques » issues de l'expérience ou de l'observation.

C'est l'opposé d'une recherche exhaustive (qui explore toutes les possibilités) : l'heuristique « triche intelligemment » en réduisant le champ des cas à examiner.

Lorsqu'une correspondance partielle est trouvée (les k dernières lettres du motif coïncident, mais pas la suivante), on peut parfois sauter plus loin que ne le suggère la table du mauvais caractère. La table du bon suffixe précalcule, pour chaque longueur de suffixe coïncidant, la prochaine position dans le motif où ce même suffixe réapparaît — ou, si ce suffixe n'apparaît nulle part ailleurs, de combien on peut décaler sans risque.

Le décalage final est le **maximum** des deux heuristiques (mauvais caractère et bon suffixe). En pratique, Boyer-Moore complet est l'un des algorithmes les plus rapides sur les textes en langage naturel, mais sa mise en œuvre est significativement plus complexe. La variante Horspool (mauvais caractère seul) est suffisante dans le cadre de l'enseignement de NSI.

5. Mais que sait faire Python nativement ?

Pour la culture : Python dispose nativement de fonctionnalités de recherche textuelle. Son implémentation en C (CPython) utilise un algorithme hybride, combinant Boyer-Moore-Horspool, un algorithme de type Sunday, le *Two-Way algorithm* ainsi que des heuristiques adaptatives selon la taille du `motif` et du `texte`.

Le tout est bien sûr codé en C et très optimisé à bas niveau. En pratique `motif in texte` ou `texte.find(motif)` sont quasi-systématiquement plus rapides que n'importe quelle implémentation Python pure, y compris une BMH bien écrite. Il est souvent inutile et peu judicieux de réimplémenter ce que Python fait déjà nativement dans des projets professionnels !